

Memoização

A memoização no React é uma técnica utilizada para otimizar o desempenho de componentes funcionais, evitando re-execuções desnecessárias de cálculos ou processamentos. Quando um componente é memoizado, o React armazena em cache o resultado da sua renderização e o reutiliza sempre que as suas dependências permanecem iguais.

A memoização é especialmente útil em situações em que um componente é renderizado várias vezes, mas as suas propriedades ou estado não foram alterados. Em vez de realizar novamente todo o processamento interno do componente, ela permite que o React simplesmente retorne o resultado armazenado em cache, economizando tempo e recursos.

No React, existem duas maneiras principais de realizar a memoização de componentes: utilizando o `React.memo` e o `useMemo`.

React.memo

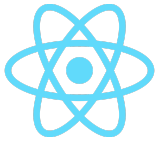
O [React.memo](#) é aplicado ao redor da definição de um componente. Ele memoiza o componente inteiro, comparando as **propriedades** recebidas para determinar se o componente precisa ser re-renderizado. `React.memo` é utilizado para memoizar componentes funcionais, garantindo que eles só sejam re-renderizados quando as propriedades forem alteradas.

```
import { memo } from 'react';

const MemoizedCard = memo(({ prop1, prop2 }) => {
  // Processamento interno do componente
  // ...

  return (
    // JSX do componente Card
    <Card prop1={prop1} prop2={prop2} />
  );
});
```

No exemplo acima, o `memo` envolve a definição do componente `MemoizedCard` e indica que ele deve ser memoizado. Assim, o React verificará se as propriedades `prop1` e `prop2` foram alteradas antes de re-renderizar o componente. Se as propriedades permanecerem as mesmas, o React utilizará o resultado da renderização anterior, evitando uma nova renderização desnecessária.



useMemo

O [useMemo](#) é um dos hooks fornecidos pelo React que permite memoizar valores calculados. Ele é usado para otimizar o desempenho de um componente, evitando cálculos repetidos e custosos sempre que o componente é renderizado.

A sintaxe básica é essa:

```
const memoizedValue = useMemo(() => {  
  // Cálculo do valor memoizado  
  return value;  
}, [dependencies]);
```

A função `useMemo` recebe dois argumentos: uma função de cálculo e uma lista de dependências. A função de cálculo é executada durante a renderização do componente e retorna o valor a ser memoizado.

A lista de dependências no `useMemo` é opcional e define quais valores devem ser considerados para decidir se o valor memoizado precisa ser recalculado. Se algum valor na lista de dependências mudar, a função de cálculo será chamada novamente para recalcular o valor memoizado.

Se nenhuma lista de dependências for passada para o `useMemo`, o comportamento será equivalente a passar uma lista de dependências vazia (`[]`). Isso significa que a função de cálculo será executada apenas uma vez, durante a renderização inicial do componente, e o valor retornado será memoizado e mantido ao longo do tempo.

A utilização do `useMemo` é recomendada quando você possui cálculos ou operações mais pesadas que dependem de valores específicos, como filtragem de dados, ordenação, formatação complexa ou chamadas de API. Ao memoizar esses valores, você pode evitar o desperdício de recursos ao evitar cálculos desnecessários em cada renderização.